

Lab5 实验报告

姓名：杨添燊

学号：24300240173

一、实验目标

- CPU 需要支持以下指令并通过测试：
实现指令：MRET, ECALL
- 实现 MMU：支持 Sv39 页表，完成虚拟地址到物理地址的翻译
- FPGA 上板测试：在 Basys3 开发板上运行并通过验证

Bonus: MMU 支持巨页

二、实现思路

1. MMU 启用条件

1.1 SATP 寄存器

SATP寄存器控制 MMU 的开启和页表基址，布局如下：

63	60 59	44 43	0

	MODE	ASID	PPN

地址翻译模式		地址空间标识符（本实验置零）	根页表物理页号

RISC-V 特权架构规定，当 SXLEN=64 时，MODE 字段的取值含义：

MODE 值	名称	含义
0	Bare	不进行地址翻译或保护
1-7	—	保留
8	Sv39	基于页的 39 位虚拟地址翻译
9	Sv48	基于页的 48 位虚拟地址翻译
10	Sv57	基于页的 57 位虚拟地址翻译
11	Sv64	基于页的 64 位虚拟地址翻译
12-15	—	保留

1.2 本项目 MMU 启用条件

本项目的 MMU 启用的条件是特权级与 satp.MODE 的联合判断（mmu.sv）：

```
assign mmu_on = (priv_mode != 2'b11) && (satp_v.mode == 4'd8);
```

M 模式下 MMU 始终关闭。无论 `satp.mode` 为何值，M 模式总是以物理地址直接访问内存。CPU 上电后初始为 M 模式，确保引导代码能以物理地址正常运行。U/S 模式下并非总是开启 MMU。如果 `satp.mode == 0`，即使 CPU 处于 U 模式，MMU 也处于直通模式，不进行地址翻译。操作系统可以选择在适当时机设置 `satp.mode = 8` 来启用分页。仅在显式配置 Sv39 时才翻译。操作系统必须在设置页表后，通过 CSR 写指令将 `satp.mode` 设为 8，MMU 才对 U/S 模式的访存进行 Sv39 翻译。

1.3 复位后的状态

SATP 寄存器在复位时初始化为 0：

```
if (reset) begin
    satp_d = 64'b0; // MODE = 0 (Bare), PPN = 0
```

复位后 `satp.mode = 0`，MMU 不启用。操作系统启动过程中需依次构建 Sv39 页表结构，将根页表物理地址写入 `satp.PPN`，将 `satp.mode` 设为 8，最后执行 `mret` 进入 U 模式；此后 MMU 才对 U 模式代码生效。

2. 整体架构与模块划分

本项目采用 MMU 挂载在 CBus Arbiter 之后的方式，无需修改 Core 内部的取指/访存接口，只需将 `priv_mode` 和 `satp` 信号从 Core 引出至 VTop，在 CBus 层面统一做地址翻译：

```
Fetch(ibus) → IBusToCBus ─┐
                           │ CBusArbiter — MMU — RAM (Verilator)
Memory(dbus)→ DBusToCBus ─┘
```

```
VTop(core) → CBusArbiter — MMU — mycpu_top — cbus_crossbar ─┐ BRAM
                                                           └─ Device(UART)
```

3. ECALL/MRET 指令实现

ECALL (0x00000073) 和 MRET (0x30200073) 均属于 SYSTEM 操作码。`decode.sv` 中通过 `funct3` 字段区分 CSR 指令与陷阱指令，`execute.sv` 通过组合逻辑识别具体陷阱类型：

```
is_ecall = id_ex_reg.is_system && !id_ex_reg.is_csr
          && (id_ex_reg.instr == 32'h00000073);
is_mret  = id_ex_reg.is_system && (id_ex_reg.instr == 32'h30200073);

trap_fire = step & id_ex_reg.valid & (is_ecall | is_mret);
```

当 `trap_fire=1` 时并行执行三项操作：

(1) 流水线冲刷与 PC 重定向：

```
redirect_pc = is_mret ? csr_mepc // MRET: 返回到保存的 PC
                : csr_mtvec;    // ECALL: 跳转到陷阱向量
```

(2) 流水线标记注入：在 `ex_mem_reg` 中设置 `trap` 标记字段：

```

ex_mem_reg.trap_pending <= 1'b1;           // 标记为陷阱指令
ex_mem_reg.trap_is_mret <= trap_is_mret; // 陷阱类型
ex_mem_reg.trap_priv    <= priv_mode_q; // 记录旧特权级（用于 mcause）
ex_mem_reg.is_load      <= 1'b0;         // 陷阱指令不产生访存
ex_mem_reg.is_store     <= 1'b0;
ex_mem_reg.csr_pending  <= 1'b0;         // 陷阱指令不写 CSR

```

(3) 陷阱标记随流水线传播：trap_pending 经 MEM → WB 逐级传递，最终在 WB 阶段触发 CSR 状态更新。

本实验的核心设计挑战在于特权级的立即生效需求与 CSR 状态延迟提交需求之间的权衡

(csr_regfile)。特权级必须在 EX 阶段立即切换，MMU 的启用与禁用依赖当前特权级，因此 MRET 降权和 ECALL 升权不能等到 WB。同时，CSR 状态必须在 WB 阶段提交，为与 NEMU 参考模型保持提交顺序一致，mcause、mepc、mstatus 的变更延迟到 WB。

在 EX 阶段（特权级立即切换）：

```

if (trap_fire_ex) begin
    if (trap_is_mret_ex)
        priv_mode_d = mstatus_q[12:11]; // MRET: 切换到 MPP (如 U-mode)
    else
        priv_mode_d = 2'b11;           // ECALL: 切换到 M-mode
end

```

此块在 trap_csr_commit 块之后执行，确保当 EX 和 WB 同时有陷阱指令时，较新的 EX 陷阱拥有更高优先级。

在 WB 阶段（CSR 状态提交）ECALL 陷阱处理：

```

// ECALL 在 WB 提交
mepc_d      = trap_pc;           // 保存当前 PC
priv_mode_d = 2'b11;             // 升权到 M-mode
mstatus_d[12:11] = trap_priv_wb; // MPP ← 旧特权级
mstatus_d[7]    = mstatus_q[3];  // MPIE ← MIE
mstatus_d[3]    = 1'b0;          // MIE ← 0 (关中断)
mcause_d = (trap_priv_wb == U) ? 8 // U-mode ecall
           : (trap_priv_wb == S) ? 9 // S-mode ecall
           : 11;                  // M-mode ecall

```

MRET 陷阱处理：

```

// MRET 在 WB 提交（不修改 mcause）
priv_mode_d = mstatus_q[12:11]; // 降权到 MPP
mstatus_d[3] = mstatus_q[7];     // MIE ← MPIE
mstatus_d[7] = 1'b1;             // MPIE ← 1
mstatus_d[12:11] = 2'b00;        // MPP ← U-mode
// Note : MRET 不修改 mcause

```

总结前文，我们以一次系统调用为例：

1. U 模式执行 ECALL
 - └ EX: trap_fire_ex → priv_mode = M (立即升权)
 - └ PC 跳转到 mtvec
 - └ trap 标记随流水线传播
 - └ WB: mepc ← ecall_pc, mcause ← 8, mstatus.MPP ← U, mstatus.MPIE ← MIE, mstatus.MIE ← 0
2. 陷阱处理程序 (M 模式) 执行
 - └ 读取 mcause, 处理系统调用, 修改 mepc 指向 ecall+4
3. 陷阱处理程序执行 MRET
 - └ EX: trap_fire_ex → priv_mode = MPP = U (立即降权)
 - └ PC 跳转到 mepc = ecall+4
 - └ WB: mstatus.MIE ← mstatus.MPIE, mstatus.MPIE ← 1, mstatus.MPP ← 0 (mcause 保持不变, 仍为 8)

4. Sv39 MMU 实现

1. 状态机设计

本项目的 MMU 实现了一个三状态 FSM (`mmu.sv:78-119`) :

```

      req_in.valid
ST_IDLE ───────────→ ST_PTE -(pte_hold)→ ST_WAIT -(resp.ready&last)→ ST_IDLE
  
```

状态	功能
ST_IDLE	等待总线请求。收到有效请求后锁存原始地址, 进入翻译状态
ST_PTE	等待 PTEHelper (Difftest 提供的页表查询模块) 给出翻译结果。pte_hold 确保等待一个周期后读取结果
ST_WAIT	输出翻译后的物理地址到总线, 等待 RAM 响应。响应到达后返回 IDLE

2. 地址翻译函数

Sv39 支持最多三级页表, 实际叶页表级别可能为 0/1/2 级。PTEHelper 返回叶页表级别 `level` 和页表项 `pte`。 `mmu.sv:65-75` 的 `sv39_paddr` 函数根据级别拼接物理地址:

```

function automatic addr_t sv39_paddr(word_t pte_val, addr_t vaddr, logic [7:0]
level);
    unique case (level)
        8'd0: sv39_paddr = {8'b0, pte_val[53:28], vaddr[29:12], vaddr[11:0]}; //
1GB 巨页
        8'd1: sv39_paddr = {8'b0, pte_val[53:19], vaddr[20:12], vaddr[11:0]}; //
2MB 巨页
        default: sv39_paddr = {8'b0, pte_val[53:10], vaddr[11:0]};
    // 4KB 普通页
    endcase
endfunction
  
```

3. 组合逻辑输出

mmu.sv:121-136 的输出逻辑根据 MMU 状态切换：

- MMU 关闭 (M 模式或 Bare 模式)：直通 `req_out = req_in, resp_out = resp_in`
- 翻译等待中 (ST_IDLE / ST_PTE)：阻止总线请求和响应 (`valid=0, ready=0`)，防止下游看到未翻译的地址
- 翻译完成 (ST_WAIT)：输出翻译后的请求地址，响应直通

4. VIVADO FPGA 路径

FPGA 上没有 PTEHelper，MMU 在 VIVADO 路径下采用恒等映射 (mmu.sv:105-106)：

```
`else // VIVADO
    paddr <= saved_req.addr; // 虚拟地址 = 物理地址
```

此解决方法的前提是，FPGA 测试程序必须使用恒等页表映射。

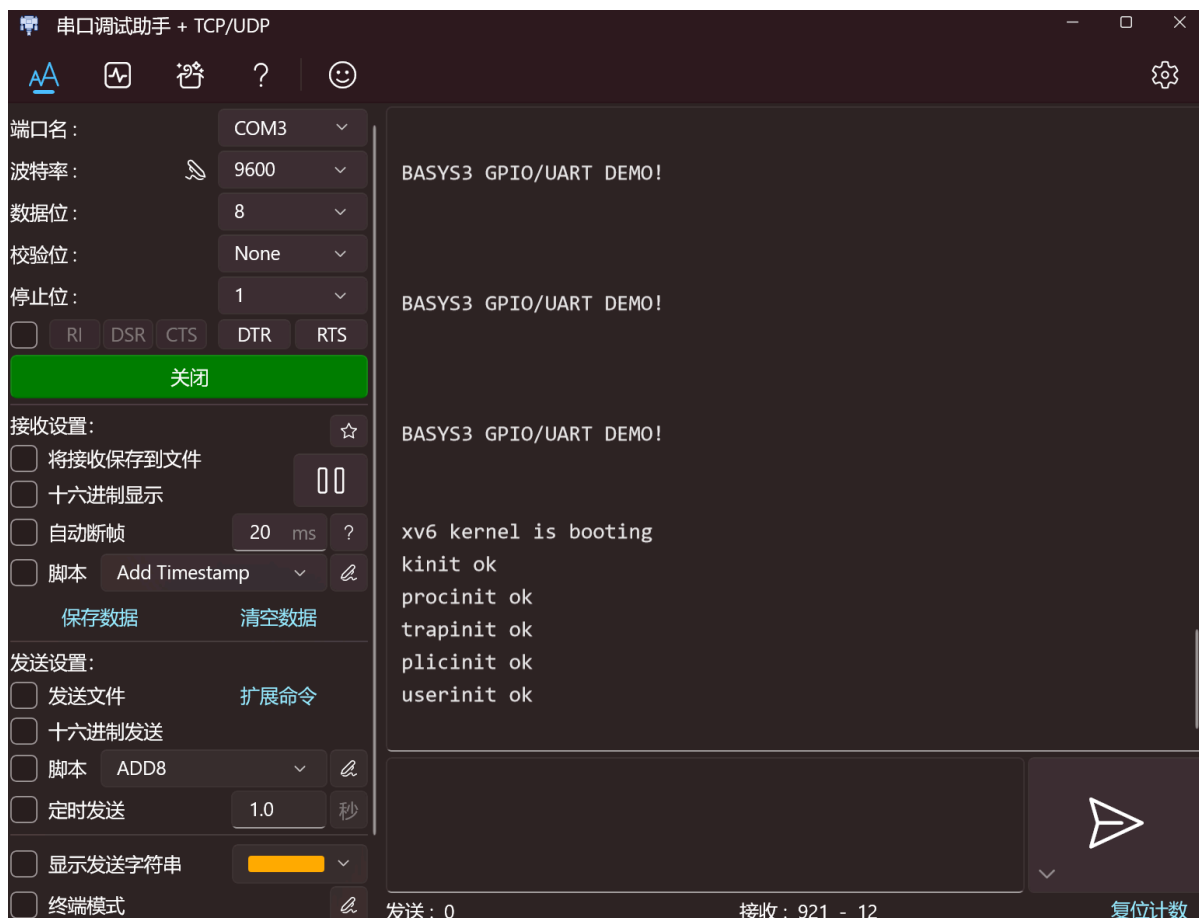
三、仿真测试

使用提供的 Makefile 运行 `make test-lab5` 进行仿真测试。输出 `Return from init! Test passed` 后 CPU 进入空闲循环（预期行为）。Difftest 全程无报错。

```
xv6 kernel is booting
[WARNING] difftest store queue overflow
kinit ok
procinit ok
trapinit ok
plicinit ok
userinit ok
Return from init! Test passed
```

四、上板测试

Vivado 流程：Run Simulation → Synthesis → Implementation → Generate Bitstream → Program Device。串口调试助手设置为 9600 波特，可正常接收测试程序输出。



五、实验中解决的问题

1.MRET 错误清零 mcause

现象: `Return from init! Test passed` 后出现 `mcause different at pc=0x80001dfc, right=8, wrong=0`

原因是RISC-V 特权架构规范规定 MRET 不修改 `mcause` 寄存器。原代码在 MRET 的 `trap_csr_commit` 路径中错误地执行了 `mcause_d = 64'b0`, 导致每次 MRET 都将 `mcause` 清零。修复时删除 MRET 路径中的 `mcause_d = 64'b0`。

2.always_comb 优先级问题

现象: 当 EX 阶段有新陷阱同时 WB 阶段有旧陷阱提交时, WB 陷阱可能覆盖 EX 陷阱的 `priv_mode_d` 设置。

修复 (`csr_regfile.sv:137-163`): 将 `trap_csr_commit` 块放在 `trap_fire_ex` 块之前, 利用 SystemVerilog `always_comb` 中后写覆盖先写的语义, 确保 EX 阶段陷阱的优先级高于 WB 阶段陷阱。

3.mmu.sv 在 Vivado 路径下是一个空操作直通, 没有真正实现 Sv39 页表遍历。具体问题 mmu.sv 的地址翻译逻辑中:

```
// 原代码:
`ifndef VERILATOR
    // 仿真环境: 用 DiffTest 提供的 PTEHelper 获取页表项, 然后翻译地址
    if (pte_pf == 8'b0)
        paddr <= sv39_paddr(pte, saved_req.addr, pte_level);
    else
        paddr <= 64'b0;
`else
    // Vivado/FPGA: 直接透传!!!
    paddr <= saved_req.addr;
`endif
```

原因是 PTEHelper 是 DiffTest 仿真框架提供的辅助模块, 在真实硬件中不存在。在 Vivado 中, MMU 直接把虚拟地址当作物理地址传出去, 没有做任何页表翻译。修复: mmu.sv:65-75 的 sv39_paddr 函数根据级别拼接物理地址 (见第二节、4小节)。

4.FPGA 串口无输出

Vivado 仿真通过, 但上板后串口无 CPU 输出。FPGA 上 CPU 运行在 cpu_clk (由 clk_wiz_0 PLL 产生), UART 外设运行在板载 clk (100MHz)。原代码中 valid/addr/wdata 直接在 clk 域的组合逻辑中使用, 存在跨时钟域 (CDC) 问题, send 脉冲可能被 clk 边沿错过:

```
// 原代码: valid 来自 cpu_clk 域, 直接在 clk 域使用
assign send = (idx != 0 && finish) || (addr == TX_DATA && valid && wvalid);
```

六、实验总结

本次实验实现了以下功能:

1. 实现指令: MRET (从陷阱返回)、ECALL (环境调用/系统调用)。
2. 特权级别切换: M 模式 (Machine mode) 与 U 模式 (User mode) 之间的切换。
3. 实现 MMU: 支持 Sv39 页表, 完成了虚拟地址到物理地址的翻译。
4. CSR 寄存器: 在 Lab4 基础上, 增加陷阱相关的 CSR 状态更新 (mstatus, mepc, mcause, mtvec 等)。
5. Verliator 仿真测试: 运行 make test-lab5 输出 Return from init! Test passed。
6. FPGA 上板测试: 在 Basys3 开发板上运行并通过了验证。